LAB2 part2 Оброблення даних про файли (робочий зошит)

Exported 01.03.2026, 10:15:20

LAB2 Частина 2. Оброблення даних про файли

Виконавець: Шевченко Микола Вікторович, група К-27
Викладач: Ляшко Андрій

Нам знадобиться бібліотека `pandas`.
Бібліотека є сторонньою, отже потребує встановлення. Краще у віртуальне оточення. Мала бути встановлена, якщо ні (клітина з імпортуванням виконується з помилкою), то треба встановити. Можна прямо з Jupyter. Тільки не під час пари.

Наступну клітину можна розкоментовувати та виконувати тільки, якщо в якості ядра (kernel) вибране віртуальне оточення!

In [1]:

```
#!pip install pandas
```

Якщо встановлювати не в оточення, то хоча б тільки для користувача (з ключем --user).

In [2]:

```
#!pip3.14 install pandas --user    # Linux
```

In [3]:

```
#!py -3.14 -m pip install pandas --user    # Windows
```

In [4]:

```
from pathlib import Path
import pandas as pd
print(f'pandas {pd.__version__}')
```

```
pandas 3.0.0
```

The document was converted with **Code-Format**: https://code-format.com/

Постановка задачі

Згадка про минуле (Лабораторну роботу №1)
Протягом навчання програмні проєкти з різних предметів зберігалися в папках з переліку `folders`, який оброблявся в лабораторній роботі №1.
Щоб профільтрувати допоміжні файли середовищ, було введено "білий список" розширень (`WHITE_LIST`) та список розширень файлів з текстом програм(задається) (`CODE_LIST`).
Також було написано функцію `kind`, що за розширенням повертає рядок з символом C,W або O відповідно для файлів з текстом програм (відповідно до переліку `CODE_LIST`), інших з білого списку, та тих, що не потрапили в білий список).

У результаті виконання лабораторної роботи для кожного місяця та кожного розширення, що зустрічаються при перегляді файлів, розміщених у папках зі списку, було знайдено таку інформацію:

`рік, місяць, розширення, кількість файлів, сумарний обсяг у байтах, кількість їх рядків.`

Кількість рядків наводиться тільки для розширень категорії C; білі та порожні рядки не рахуються; для інших розширень ця клітина таблиці залишається незаповненою.

За цією інформацією було побудовано csv-файл з іменем `lab1_1.csv` (кодування тільки utf-8), на початку якого розміщено рядок з заголовками: year, mon, ext, cnt, size, lines.

In [5]:

```
CODE_LIST = {'.cpp', '.hpp', '.c', '.h', '.py', '.pyw', '.cs', '.js',
'.java'}
WHITE_LIST = CODE_LIST | {'.html', '.json', '.txt', '.ipynb'}
```

In [6]:

```
headers = ['year', 'mon', 'ext', 'cnt', 'size', 'lines']
```

The document was converted with **Code-Format**: https://code-format.com/

За інформацією з csv-файлу з'ясувати

1. Скільки різних розширень зустрічається у папках з заданого списку, скільки файлів з кожним розширенням. (Для цієї задачі розглядаємо всі розширення, а не тільки "білого списку".)
   Відповідну інформацію вивести на екран у табличному вигляді. Рядки таблиці містять розширення, кількість файлів та ознаку (C,W,O відповідно для файлів з текстом програм (розширення з списку `CODE_LIST`), інших з білого списку, та тих, що не потрапили в білий список). Інформація подається за спаданням кількості файлів.
2. За датами створення файлів для кожного місяця, починаючи з вересня 2024-го року, знайти таку інформацію: кількість файлів з розширеннями з "білого списку", їх сумарний розмір у кілобайтах, кількість їх рядків (враховувати тільки для файлів категорії C), середній розмір у байтах, середню кількість рядків.
   Відповідну інформацію вивести на екран у табличному вигляді в порядку, оберненому до хронологічного.

Зверніть увагу на підписи стовпчиків, що будуть виводитись.

Всі значення округлити та привести до цілого типу.

Завантаження даних

Задаємо шлях до файлу `lab1_1.csv`.

In [7]:

```
path = Path("lab1_1.csv")
```

Завантажимо дані за допомогою функції `pd.read_csv` (документація тут).

Зверніть увагу на те, що:

1. файл записано в кодуванні `utf-8`
2. перший рядок файлу містить заголовки year, mon, ext, cnt, size, lines
3. значення у стовпчику lines можуть бути відсутні

▶ Підказки.

▶ Ще більше підказок

The document was converted with **Code-Format**: https://code-format.com/

In [8]:

```
data = pd.read_csv(path, encoding = 'utf-8', header=0, sep=",", dtype=
{"lines": "Int64"}, names=headers)
data.head(3)
```

		year	mon	ext	cnt	size	lines
2025	**1**	.cpp	1	929	33.0	929	33
	1	.h	2	2090	103.0	1045	51
	1	.json	2	5086	NaN	2543	<NA>

In [9]:

```
data.tail(3)
```

		year	mon	ext	cnt	size	lines
2025	**12**	.cs	7	6512	217.0	930	31
	12	.json	4	15008	NaN	3752	<NA>
	12	.txt	1	414	NaN	414	<NA>

Подивимось інформацію про завантажені дані. Документація на метод `info`.

The document was converted with **Code-Format**: https://code-format.com/

In [10]:

```
data.info()
```

```
<class 'pandas.DataFrame'>
MultiIndex: 30 entries, (np.int64(2025), np.int64(1)) to
(np.int64(2025), np.int64(12))
Data columns (total 6 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   year    30 non-null     str
 1   mon     30 non-null     int64
 2   ext     30 non-null     int64
 3   cnt     18 non-null     float64
 4   size    30 non-null     int64
 5   lines   18 non-null     Int64
dtypes: Int64(1), float64(1), int64(3), str(1)
memory usage: 1.6 KB
```

Бачимо, що відсутні значення є не тільки в стовпчику `lines`, що природньо, бо кількість рядків обчислювалась не для всіх розширень, але і в стовпчику `ext` з розширеннями.

Замінимо відсутні розширення порожнім рядком.

Використаємо метод `fillna`.

Слід замінити тільки відсутні розширення, тому значення параметра `value` слід зробити словником, що для кожного стовпчика, який слід оброблювати, задає значення-заповнювач.
Не забудьте, що перетворення слід виконувати "на місці".

▶ Підказка (тільки за повного безвихіддя)

Альтернатива. Можна спробувати перетворити тільки стовпчик `data.ext`.
▶ Підказка (тільки за повного безвихіддя)

Перший спосіб більш економний за пам'яттю. Другий спосіб корисний тоді, коли для різних стовпчиків використовують різні алгоритми заповнення відсутніх значень.

The document was converted with **Code-Format**: https://code-format.com/

In [11]:

```
data.fillna(value={'ext': ''}, inplace=True)
data.head(3)
```

		year	mon	ext	cnt	size	lines
2025	**1**	.cpp	1	929	33.0	929	33
	1	.h	2	2090	103.0	1045	51
	1	.json	2	5086	NaN	2543	<NA>

In [12]:

```
data.info()
```

```
<class 'pandas.DataFrame'>
MultiIndex: 30 entries, (np.int64(2025), np.int64(1)) to
(np.int64(2025), np.int64(12))
Data columns (total 6 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   year    30 non-null     str
 1   mon     30 non-null     int64
 2   ext     30 non-null     int64
 3   cnt     18 non-null     float64
 4   size    30 non-null     int64
 5   lines   18 non-null     Int64
dtypes: Int64(1), float64(1), int64(3), str(1)
memory usage: 1.6 KB
```

Тепер незаповнені дані залишились тільки у стовпчику `lines`.

`Ні в якому разі там не повинно бути нулів!`

Групування та групові операції

The document was converted with **Code-Format**: https://code-format.com/

In [13]:

```
data.head()
```

		year	mon	ext	cnt	size	lines
2025	**1**	.cpp	1	929	33.0	929	33
	1	.h	2	2090	103.0	1045	51
	1	.json	2	5086	NaN	2543	<NA>
	2	.cpp	7	5860	202.0	837	28
	2	.h	4	1619	66.0	404	16

Задача 1 Скільки різних розширень зустрічається в папках з заданого списку, скільки файлів з кожним розширенням?

Відповідну інформацію вивести на екран у табличному вигляді. Рядки таблиці містять розширення, кількість файлів та ознаку (C,W,O відповідно для файлів з текстом програм (розширення з списку `CODE_LIST`), інших з білого списку, та тих, що не потрапили в білий список). Інформація подається за спаданням кількості файлів.

Скільки різних розширень зустрічається в папках з заданого списку?

Найпростіше: взяти стовпчик таблиці, що зберігає розширення, та знайти кількість унікальних значень.

▶ Підказки

In [14]:

```
data.ext.nunique()
```

```
29
```

Скільки файлів з кожним розширенням?

Відповідну інформацію вивести на екран у табличному вигляді. Рядки таблиці містять розширення, кількість файлів та ознаку (C,W,O відповідно для файлів з текстом програм (розширення з списку `CODE_LIST`), інших з білого списку, та тих, що не потрапили в білий список). Інформація подається за спаданням кількості файлів.

The document was converted with **Code-Format**: https://code-format.com/

Спочатку треба підсумувати інформацію по кожному розширенню. Допоможе метод `groupby` [документація тут](#).

Групувати слід за розширенням (параметр `by`) і для кожної групи обчислити суму кількостей файлів (метод `sum`).

Щоб не сумувати роки, місяці тощо, групування слід застосувати до вибірки потрібних стовпчиків. Маємо вибрати стовпчик, за яким групуємо, і стовпчик, який сумуємо.

▶ Підказка

Результат присвоїти змінній `task1`.

The document was converted with **Code-Format**: https://code-format.com/

In [15]:

```
task1  = data[["ext", "cnt"]].groupby(by=["ext"]).sum()
task1
```

The document was converted with **Code-Format**: https://code-format.com/

	cnt
ext	
305	15.0
414	0.0
929	33.0
1088	44.0
1619	66.0
2090	172.0
2391	85.0
2866	0.0
4683	141.0
5086	0.0
5340	185.0
5860	202.0
6512	217.0
7200	262.0
11182	425.0
12224	0.0
14331	386.0
14417	476.0
15008	0.0
20094	0.0
25664	0.0
25692	0.0
38062	1238.0
53347	0.0
172659	3471.0

The document was converted with **Code-Format**: https://code-format.com/

	cnt
ext	
1194799	0.0
2499139	0.0
2571027	0.0
5033053	92615.0

Відсортувати за спаданням кількості файлів. Допоможе метод `sort_values` ([документація тут](#)).

Не забудьте вказати, що сортування відбувається за спаданням та "на місці".

▶ Підказки

In [16]:

```
task1.sort_values(by='cnt', ascending=False, inplace=True)
task1.head(3)
```

	cnt
ext	
5033053	92615.0
172659	3471.0
38062	1238.0

За умовою ще треба додати стовпчик з ознакою категорії розширення файлу W,C,O (див. умову). У лабораторній роботі №1 була написана функція `kind`, що за розширенням повертає рядок з шуканою ознакою. Використаємо її.

In [17]:

```
def kind(extension):   # записати
    if extension in CODE_LIST:
        return 'C'
    elif extension in WHITE_LIST:
        return 'W'
    else:
        return '0'
```

The document was converted with **Code-Format**: https://code-format.com/

Додамо до таблиці стовпчик з ознаками розширень файлів.

Зараз розширення є індексом. За допомогою методу метод `reset_index` (документація тут) зробимо його стовпчиком таблиці.

Не забувайте виконувати перетворення "на місці".

In [18]:

```
task1.reset_index(inplace=True)
task1.head(3)
```

	ext	cnt
0	5033053	92615.0
1	172659	3471.0
2	38062	1238.0

Тепер вже додаємо стовпчик з ознаками.

▶ Підказки

In [19]:

```
task1['kind'] = task1.ext.apply(kind)
task1.head(3)
```

	ext	cnt	kind
0	5033053	92615.0	0
1	172659	3471.0	0
2	38062	1238.0	0

За допомогою методу `set_index` (документація тут) знов зробимо розширення індексом таблиці.

Тут: Значенням параметра `keys` має стати ім'я стовпчика, що слід використати як індекс.

Не забувайте виконувати перетворення "на місці".

The document was converted with **Code-Format**: https://code-format.com/

In [20]:

```
task1.set_index(keys='ext', inplace=True)
task1.head(3)
```

	cnt	kind
ext		
5033053	92615.0	0
172659	3471.0	0
38062	1238.0	0

Змінимо написи.

In [21]:

```
task1.index.name='Розширення'
task1.columns=['Кількість файлів','Ознака']
task1.head()
```

	Кількість файлів	Ознака
Розширення		
5033053	92615.0	0
172659	3471.0	0
38062	1238.0	0
14417	476.0	0
11182	425.0	0

The document was converted with **Code-Format**: https://code-format.com/

In [22]:

```
task1.head(20)
```

	Кількість файлів	Ознака
Розширення		
5033053	92615.0	0
172659	3471.0	0
38062	1238.0	0
14417	476.0	0
11182	425.0	0
14331	386.0	0
7200	262.0	0
6512	217.0	0
5860	202.0	0
5340	185.0	0
2090	172.0	0
4683	141.0	0
2391	85.0	0
1619	66.0	0
1088	44.0	0
929	33.0	0
305	15.0	0
5086	0.0	0
2866	0.0	0
414	0.0	0

The document was converted with **Code-Format**: https://code-format.com/

Задача 2 За датами створення файлів для кожного місяця знайти таку інформацію: кількість файлів з розширеннями з "білого списку", їх сумарний розмір у кілобайтах, кількість їх рядків (враховувати тільки для файлів категорії C), середній розмір у байтах, середню кількість рядків.

Відповідну інформацію вивести на екран у табличному вигляді в порядку, оберненому до хронологічного.

Всі значення округлити та привести до цілого типу.

План:

1. Додати до початкової таблиці `data` стовпчик `kind`, що містить ознаки розширень.
2. Вибрати дані, що стосуються розширень з "білого списку".
3. Згрупувати вибрані дані в межах року та місяця.
4. За згрупованою інформацією обчислити потрібні агрегати.
5. Виконати перетворення стовпчиків та відсортувати.

Щоб за невдачі повторно не завантажувати дані, створимо їх копію.

In [23]:

```
data1 = data.copy()
```

1. Додати до початкової таблиці `data` стовпчик `kind`, що містить ознаки розширень.

Аналогічно тому, що виконувалось раніше, додати до таблиці `data1` стовпчик `kind`, що містить ознаки.

In [24]:

```
data1['kind'] = data1.ext.apply(kind)
data1.head(3)
```

		year	mon	ext	cnt	size	lines	kind
2025	**1**	.cpp	1	929	33.0	929	33	0
	1	.h	2	2090	103.0	1045	51	0
	1	.json	2	5086	NaN	2543	<NA>	0

The document was converted with **Code-Format**: https://code-format.com/

2. Вибрати дані, що стосуються розширень з "білого списку".

Запишіть вираз, що задає вибірку тільки інформації про розширення з "білого списку". Щоб далі не копіювати сам вираз, присвойте його значення змінній `selection`. (Це є зручно, коли експериментуємо, у продакшн потреби в цій змінній насправді не буде, замість неї можна буде використовувати вираз безпосередньо.)

▶ Підказка

In [25]:

```
selection = data1[data1.kind != 'C']
selection.head()
```

		year	mon	ext	cnt	size	lines	kind
2025	**1**	.cpp	1	929	33.0	929	33	0
	1	.h	2	2090	103.0	1045	51	0
	1	.json	2	5086	NaN	2543	<NA>	0
	2	.cpp	7	5860	202.0	837	28	0
	2	.h	4	1619	66.0	404	16	0

3. Згрупувати вибрані дані за ознаками в межах року та місяця.

   Допоможе метод `groupby` [документація тут](#).

Групувати слід за роком та місяцем (параметр `by` має бути списком). Оскільки далі будемо обчислювати статистики для розміру та кількості рядків, то перед групуванням з таблиці слід вибрати стовпчики, за якими групуємо і за якими обчислюємо статистики.

In [26]:

```
task2 = selection[['year', 'mon', 'cnt', 'size', 'lines']].groupby(by=
['year', 'mon'])
```

The document was converted with **Code-Format**: https://code-format.com/

Далі необхідно знайти кількість файлів за кожен місяць, їх сумарний та середній розмір, а також сумарну та середню кількість рядків.

У результаті групування отримався об'єкт `DataFrameGroupBy` (документація тут).

До нього застосовний метод `agg`, що обчислює статистики (агрегатні характеристики) (документація тут).

Для стовпчика `cnt` будемо шукати суму значень (`sum`), для стовпчика `size` будемо шукати суму значень та середнє значення (`mean`), для стовпчика `lines` будемо шукати суму значень та середнє значення.

Першим параметром методу `agg` зробимо словник, що задає які статистики для яких стовпчиків слід обчислювати. Ключами словника мають бути імена стовпчиків (рядки), а значеннями списки статистик або окремі статистики (статистики також задаємо рядками-іменами)).

Результат агрегування зробимо значенням змінної `res2`.

In [27]:

```
res2 = task2.agg({'cnt': 'sum', 'size': ['sum', 'mean'], 'lines':
['sum', 'mean']})
res2.head()
```

		cnt	size		lines	
		sum	sum	mean	sum	mean
year	**mon**					
.cpp	**1**	118.0	3320	1660.0	118	59.0
	3	185.0	1780	1780.0	61	61.0
	6	811.0	4251	2125.5	134	67.0
	7	678.0	2896	1448.0	96	48.0
	11	1238.0	3460	3460.0	112	112.0

Оскільки відбувалось групування по кільком стовпчикам та для окремих стовпчиків обчислювалось кілька характеристик, то отримали складені індекси (multindex) і рядків, і стовпчиків.

Поточний індекс стовпчиків.

The document was converted with **Code-Format**: https://code-format.com/

In [28]:

```
res2.columns
```

```
MultiIndex([(  'cnt',  'sum'),
            ( 'size',  'sum'),
            ( 'size', 'mean'),
            ('lines',  'sum'),
            ('lines', 'mean')],
           )
```

Перейдемо до простого індексу стовпчиків.

In [29]:

```
res2.columns = res2.columns.to_flat_index()
res2.columns
```

```
Index([   ('cnt', 'sum'),   ('size', 'sum'),  ('size', 'mean'),
        ('lines', 'sum'), ('lines', 'mean')],
      dtype='object')
```

In [30]:

```
res2.head(3)
```

		(cnt, sum)	(size, sum)	(size, mean)	(lines, sum)	(lines, mean)
year	**mon**					
.cpp	**1**	118.0	3320	1660.0	118	59.0
	3	185.0	1780	1780.0	61	61.0
	6	811.0	4251	2125.5	134	67.0

Змінимо назви стовпчиків.

The document was converted with **Code-Format**: https://code-format.com/

In [31]:

```
res2.columns=['Кількість', 'Сумарний розмір', 'Середній розмір',
'Кількість рядків', 'Середня кількість рядків',]
res2.head(3)
```

		Кількість	Сумарний розмір	Середній розмір	Кількість рядків	Середня кількість рядків
year	**mon**					
.cpp	**1**	118.0	3320	1660.0	118	59.0
	3	185.0	1780	1780.0	61	61.0
	6	811.0	4251	2125.5	134	67.0

Виразимо сумарний розмір у kb.

Для цього слід поділити значення на 1024, округлити. Додатково перетворимо на цілий тип (метод `astype` дозволяє виконувати перетворення типів, [документація тут](#)). Також не забудемо змінити підпис стовпчика.

Переходимо до кілобайтів.

In [32]:

```
res2['Сумарний розмір'] = (res2['Сумарний розмір'] /
1024).round().astype(int)
```

Змінюємо підпис стовпчиків

In [33]:

```
res2.columns = res2.columns=['Кількість', 'Сумарний розмір, kb',
'Середній розмір, b', 'Кількість рядків', 'Середня кількість рядків',]
```

The document was converted with **Code-Format**: https://code-format.com/

In [34]:

```
res2.head(3)
```

		Кількість	Сумарний розмір, kb	Середній розмір, b	Кількість рядків	Середня кількість рядків
year	**mon**					
.cpp	**1**	118.0	3	1660.0	118	59.0
	3	185.0	2	1780.0	61	61.0
	6	811.0	4	2125.5	134	67.0

Округлити середній розмір та середню кількість рядків, перейти до цілих значень.

In [35]:

```
res2.info()
```

```
<class 'pandas.DataFrame'>
MultiIndex: 21 entries, ('.cpp', np.int64(1)) to ('.txt',
np.int64(163))
Data columns (total 5 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Кількість                 21 non-null     float64
 1   Сумарний розмір, kb       21 non-null     int64
 2   Середній розмір, b        21 non-null     float64
 3   Кількість рядків          21 non-null     Int64
 4   Середня кількість рядків  12 non-null     Float64
dtypes: Float64(1), Int64(1), float64(2), int64(1)
memory usage: 1.3+ KB
```

Ось тільки тепер замінимо відсутні значення кількості рядків нулями!

Якщо б це було зроблено раніше, то середня кількість рядків була б обрахована неправильно (на неї вплинули б нульові значення замість відсутніх).

▶ Підказка

The document was converted with **Code-Format**: https://code-format.com/

In [36]:

```
res2.fillna(0, inplace=True)
res2.info()
```

```
<class 'pandas.DataFrame'>
MultiIndex: 21 entries, ('.cpp', np.int64(1)) to ('.txt',
np.int64(163))
Data columns (total 5 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Кількість                 21 non-null     float64
 1   Сумарний розмір, kb       21 non-null     int64
 2   Середній розмір, b        21 non-null     float64
 3   Кількість рядків          21 non-null     Int64
 4   Середня кількість рядків  21 non-null     Float64
dtypes: Float64(1), Int64(1), float64(2), int64(1)
memory usage: 1.3+ KB
```

Тепер можна перетворювати типи стовпчиків.

In [37]:

```
fname = 'Середній розмір, b'
res2[fname] = res2[fname].round().astype(int)

fname = 'Середня кількість рядків'
res2[fname] = res2[fname].round().astype(int)
```

The document was converted with **Code-Format**: https://code-format.com/

In [38]:

```
res2
```

		Кількість	Сумарний розмір, kb	Середній розмір, b	Кількість рядків	Середня кількість рядків
year	**mon**					
.cpp	**1**	118.0	3	1660	118	59
	3	185.0	2	1780	61	61
	6	811.0	4	2126	134	67
	7	678.0	3	1448	96	48
	11	1238.0	3	3460	112	112
.cs	**7**	217.0	1	930	31	31
	58	3471.0	3	2976	59	59
.h	**1**	15.0	0	305	15	15
	2	172.0	2	1045	85	42
	3	185.0	2	962	61	30
	4	328.0	2	1102	81	40
.java	**497**	92615.0	10	10126	186	186
.json	**2**	0.0	4	1988	0	0
	4	0.0	7	3404	0	0
	6	0.0	7	3813	0	0
	12	0.0	2	2141	0	0
	16	0.0	3	3334	0	0
	31	0.0	38	38541	0	0
.txt	**1**	0.0	0	414	0	0
	9	0.0	279	285669	0	0
	163	0.0	15	15332	0	0

The document was converted with **Code-Format**: https://code-format.com/

Відсортуємо в порядку, оберненому до хронологічного.

У даному випадку слід сортувати індекс. Допоможе метод `sort_index` ([документація тут](#)).

The document was converted with **Code-Format**: https://code-format.com/

In [39]:

```
res2.sort_index(ascending=[False, False], inplace=True)
res2
```

		Кількість	Сумарний розмір, kb	Середній розмір, b	Кількість рядків	Середня кількість рядків
year	**mon**					
.txt	**163**	0.0	15	15332	0	0
	9	0.0	279	285669	0	0
	1	0.0	0	414	0	0
.json	**31**	0.0	38	38541	0	0
	16	0.0	3	3334	0	0
	12	0.0	2	2141	0	0
	6	0.0	7	3813	0	0
	4	0.0	7	3404	0	0
	2	0.0	4	1988	0	0
.java	**497**	92615.0	10	10126	186	186
.h	**4**	328.0	2	1102	81	40
	3	185.0	2	962	61	30
	2	172.0	2	1045	85	42
	1	15.0	0	305	15	15
.cs	**58**	3471.0	3	2976	59	59
	7	217.0	1	930	31	31
.cpp	**11**	1238.0	3	3460	112	112
	7	678.0	3	1448	96	48
	6	811.0	4	2126	134	67
	3	185.0	2	1780	61	61
	1	118.0	3	1660	118	59

The document was converted with **Code-Format**: https://code-format.com/

Частину 2 зроблено!

The document was converted with **Code-Format**: https://code-format.com/